

SENTINEL

An Autonomous Incident-Response Agent

A complete, from-scratch study guide

The Complete Study Guide

Everything in this project, explained from scratch — in plain English, with analogies, and with the exact words to use when you explain it out loud. Read it cold; teach it after.

What it is	An AI agent that investigates production incidents and proposes fixes for a human to approve.
Core stack	LangGraph · LangChain · MCP · QLoRA fine-tuning · pgvector RAG · FastAPI · Streamlit
The headline	Fine-tuned a 1.5B model to 100% category accuracy — then proved with a harder test that it had learned patterns, not reasoning.

Contents

SENTINEL

PART 1

The Big Picture — what it is and why it exists

The lead to reach the goal

PART 2

The Flow — what happens, step by step

An Autonomous Incident-Response Agent

PART 3

Every Concept, From Scratch

A complete, from-scratch study guide

4 Structured output

5 RAG and hybrid retrieval

6 The critic / multi-agent pattern

7 MCP — the Model Context Protocol

8 Fine-tuning with QLoRA

9 Evaluation — the part most projects skip

10 Prompt-injection defence

11 Model routing by tier

12 Observability

PART 4

The Results — and the honest story

The numbers, and why 100% was a warning sign

PART 5

How to Explain It

Resume bullets, tech stack, interview Q&A;

PART 6

Glossary — every term in one place

The Big Picture

The 30-second pitch

Sentinel is an AI agent that does what a sleepy on-call engineer does at 3 a.m. when a pager goes off. An alert fires — say, *checkout is slow*. Sentinel reads the alert, decides which logs and metrics to pull, looks at what changed recently, forms a theory of what broke, **argues against its own theory**, and if the theory holds up, writes a step-by-step fix. Then it stops and asks a human to approve before it touches anything. A person clicks approve; it executes the fix and checks whether the system recovered; then it writes a short post-mortem and remembers the lesson for next time.

The one sentence to memorise

“It’s an autonomous agent that investigates production incidents end-to-end, but it physically cannot change anything without a human approving first — safety is built into the structure, not asked for politely.”

What problem does it solve?

When something breaks in production, the slow and expensive part is not fixing it — it’s the **investigation**. A human has to figure out which of a hundred services is actually the culprit, dig through logs, correlate a deploy against a latency spike, and rule out red herrings. That takes minutes to hours, and every minute is downtime. Sentinel automates the investigation and hands a human a well-reasoned plan to approve — turning “figure it out from scratch” into “check this and click yes.”

The analogy that makes it click

Think of a **hospital emergency room**. A patient arrives (the alert). A triage nurse rates how urgent it is (triage). A doctor orders tests — blood work, X-rays (the read-only tools: logs, metrics, deploys). The doctor forms a diagnosis (hypothesis), and a second senior doctor challenges it to catch mistakes (the critic). They agree on a treatment (the plan) — but a controlled drug requires a second signature before it’s administered (the human approval gate). After treatment, they check the patient recovered (verify) and write it up (post-mortem). Sentinel is that ER, for software.

Why the simulated environment is a feature, not a shortcut

The “production system” Sentinel watches is faked in code. That is **deliberate**: because we control the fake world, we know the true cause of every incident — the answer key. That is what makes it possible to *measure* whether the agent was actually right, instead of just plausible. Against real infrastructure you have no answer key, so you can never score the agent. Same tool shapes as Prometheus/Loki/Kubernetes; swapping in real backends is a plumbing detail.

The five things it strings together

- **An agent brain** (LangGraph) that decides what to do next, can loop back to think more, and can pause.
- **Tools** it can call to look at the world (logs, metrics, deploys) and — only with permission — change it.

- **A knowledge base** (RAG) of runbooks it searches for relevant advice.
- **A small custom-trained model** (fine-tuned) that does the cheap, high-volume classification.
- **A safety layer** that makes destructive actions impossible without a human.

The Flow, Step by Step

Everything Sentinel does is a path through ten stages. Here is the whole journey in order. The two special moves — the **loop** and the **pause** — are what make it an agent rather than a script.

1

Triage A fast classifier reads the alert and tags it: how severe (P1–P4), what kind of failure, which service, does a human need paging now? This is the job the fine-tuned model does.

2

Retrieve Search the runbook library for advice relevant to this alert (this is RAG).

3

Investigate An agent with READ-ONLY tools pulls logs, metrics and deploy history. It decides what to look at — it is not following a fixed script.

4

Synthesize Turn the gathered evidence into a single root-cause hypothesis, as structured data.

5

Critique A second agent attacks that hypothesis: is every claim backed by evidence? Could the blamed service be a victim? Correlation or causation?



Loop If the critic is unconvinced, go BACK to Investigate with specific new questions. Capped at 4 rounds — a confused agent doesn't improve by spinning.

6

Plan Once the hypothesis holds, write a concrete remediation plan: which action, on which target, why, how risky, how to undo it.



PAUSE The graph HALTS here. A human sees the plan and clicks approve or reject. The agent cannot proceed on its own. This is the safety gate.

7

Execute Only after approval: run the approved steps exactly, no model improvising.

8

Verify Check the metrics actually recovered. A service that was merely restarted but will fail again is NOT resolved.

9

Post-mortem Write a blameless write-up and save the lesson to long-term memory, so the next similar incident starts smarter.

The two moves that make it “agentic”

The loop (■) — the system can decide it hasn't thought hard enough and go gather more evidence. A normal program runs top to bottom; an agent can revisit. **The pause** (■) — the system can stop mid-run, wait for a human, and resume later, even on a different machine, because its state is saved to a database.

Every Concept, From Scratch

Twelve ideas. Each one follows the same shape: **what it is** in plain words, an **analogy**, **why it matters**, **what we built**, and **the gotcha** — the non-obvious thing that trips people up and that you can mention to sound like you actually did it.

1

Tool calling / the agentic loop

the beating heart of every agent

What it is. You hand the model a menu of tools, each with a description. Instead of answering from memory, the model replies “call `query_Logs` with these arguments.” Your code runs that, hands back the result, and the model decides the next move. Repeat until it stops asking.

Analogy. A detective who can’t solve the case from the armchair. They send for the autopsy report, read it, then decide to interview a witness based on what it said. Each result shapes the next request.

Why it matters. This is the exact line between a chatbot and an agent. A chatbot talks; an agent *acts*. The model doesn’t know your logs — it decides to go and look.

What we built. We wrote the loop by hand rather than importing a ready-made one, because the loop *is* the concept. It runs several tool calls at once when they’re independent, times each one out, and feeds tool *errors* back to the model as observations — a good agent recovers from a bad argument by trying again.

The gotcha

Tool descriptions are part of the prompt. The model picks tools by reading those docstrings, so ours say not just *what* a tool does but *when to reach for it* and *how to read the result*. Vague tool descriptions are the #1 reason agents call the wrong tool.

2

LangGraph — state machines for agents

the orchestration layer

What it is. Instead of one big loop, you model the agent as a **graph**: boxes (nodes) are steps, arrows (edges) are transitions, and a shared “state” object flows through.

Analogy. A board game. Each square does something; some squares say “go back three spaces” (a loop); one square says “wait for the other player” (the pause). The rules are printed on the board, not hidden in your head.

Why not just a loop? Real workflows branch, cycle, and pause. A plain loop buries those in tangled if-statements. In a graph they’re *declared* — you can see the whole control flow on one screen.

Four sub-ideas worth naming separately:

Typed state + reducers

State is a typed dictionary where each field has a merge rule. Evidence is set to *add*, so investigation rounds accumulate instead of overwriting. THE classic bug: a missing merge rule silently throws away half your data — no error, just wrong results.

Conditional edges	After the critic runs, a routing function chooses the next node: loop back, or move on.
Cycles	investigate → synthesize → critique → investigate. The reflection loop. Bounded, because iterating doesn't cure confusion.
Checkpointing + interrupts	The whole state saves to Postgres at each step, so the graph can pause before the approval node, the process can exit, and someone resumes tomorrow from another machine.

3

Human-in-the-loop, done properly

the safety story

The naive version. Ask the model “should I get approval?” and trust its answer. That’s theatre — the model can just say no.

What we built — three independent walls:

- **The interrupt.** The graph structurally halts before the approval node. It cannot pass.
- **The approval flag lives outside the model’s reach** — it’s set in program memory by the graph AFTER a human decides. There is no prompt field or tool argument the model could use to set it.
- **The investigator gets a read-only tool list** — it literally does not have a destructive tool to call, whatever it concludes.

The line to remember

Confidence is not authorization. A model being sure it’s right does not make it right, and certainty is never permission to change production. Safety is structural — the agent can propose a rollback, but it has no mechanism to grant itself the right to run one.

4

Structured output

making models return data, not prose

What it is. Forcing the model’s answer into a strict shape (a schema) — specific fields with specific types — instead of free-form text.

Analogy. A form with labelled boxes versus a blank sheet. If a colleague fills in boxes, the next person can process it. If they write a paragraph, someone has to re-read and interpret it every time.

Why it matters. Free text between agent steps is where these systems rot. If step A returns prose, step B has to re-parse it, and a small wording change silently breaks a later branch.

What we built. Small and free models often break strict JSON mode — markdown fences, trailing commas, prose wrapped around the answer. So our helper escalates: ask with the schema → salvage the JSON out of whatever came back → **show the model its own validation errors and ask again** → fall back to a safe default.

The gotcha

Step three is the clever one. Telling the model “field ‘severity’ must be one of P1–P4” is a far better repair instruction than “try again.” You feed the machine its exact mistake.

5

RAG & hybrid retrieval

giving the model a library to consult

What it is. RAG = Retrieval-Augmented Generation. Before the model answers, fetch the most relevant documents and paste them into the prompt. The model “reads the manual” instead of relying on memory.

Analogy. An open-book exam. You don’t memorise the textbook; you look up the right page at the right moment.

The key upgrade — using TWO search methods together:

- **Vector search** understands meaning. Ask for “exit code 137” and it finds a section titled “memory exhaustion” even with no shared words.
- **Keyword search (BM25)** nails rare exact strings — x509, HikariPool-1, MISCONF. Meaning-based search BLURS these, because they’re near-unique tokens with little semantic content.

We run both and fuse the rankings (a method called Reciprocal Rank Fusion) rather than trying to average their scores — the two scoring systems aren’t on the same scale, and forcing them onto one requires tuning that silently goes stale.

Two smaller tricks

Chunking on headings, not fixed character counts — a runbook section is already one coherent unit of advice. **Contextual chunks** — each piece carries its heading path (“Database errors > Fix”), which makes its meaning-fingerprint far more specific.

6

The critic / multi-agent pattern

an agent that attacks its own work

What it is. A second agent whose only job is to find what’s *wrong* with the first agent’s conclusion.

Analogy. A newspaper. One journalist writes the story; a sceptical editor tears it apart before it prints. The editor’s job isn’t to say “nice article” — it’s to catch what’s unsupported.

The crucial prompt choice. We ask the critic *what is wrong* with the hypothesis, not *whether it’s reasonable*. A model asked “is this good?” says yes. It’s told to check: is every claim tied to a real observation? Is this correlation or causation? Could the blamed service be a *victim* of the real cause?

Why it matters

Acting on a wrong diagnosis during an outage makes the outage longer. In real runs the critic regularly rejected the first hypothesis and sent the agent back with *specific, answerable questions* — not a vague “look harder.”

7

MCP — the Model Context Protocol

a universal plug for AI tools

What it is. An open standard for exposing tools and data to any AI system. The common phrase: “USB-C for AI tools.”

Analogy. Before USB, every device had its own connector. USB meant any device worked with any computer. MCP is that standard plug — expose your tools once and any MCP-speaking AI (Claude Desktop, another team’s agent) can use them without importing your code.

What we built — both ends. A **server** that publishes our tools using all three MCP building blocks: tools (the model calls them), resources (read-only context addressed by a URL, like `incident://current/alert`), and prompts (reusable templates). And a **client** that loads other people’s MCP servers from a config file.

The security decision

Over MCP we expose remediation only as `propose_remediation`, which records a suggestion and runs nothing. An outside client can’t prove a human approved anything, so actual execution stays locked inside our graph behind the approval gate.

8

Fine-tuning with QLoRA

training a small model for one job

What it is. Taking a small general model and specialising it for one narrow task by showing it many examples.

The two acronyms, decoded:

- **LoRA** — instead of retraining the whole (huge) model, freeze it and train tiny “adapter” patches — about 1% of the parameters. Cheap and fast.
- **QLoRA** — the Q is “quantised.” Load the frozen model in 4-bit (low precision) to shrink it, and train the adapters on top. This is what let a 1.5-billion-parameter model train on a free Colab GPU.

Analogy. Hiring a smart generalist and giving them one week of training on your company’s specific ticket system. You don’t re-educate them from birth; you add a thin layer of job-specific skill.

When it’s the right tool. Narrow, high-volume, fixed-output jobs. Alert triage is exactly that: it runs on every alert, outputs a fixed form, and needs pattern-recognition, not deep reasoning.

The choices that mattered

Train on the answer only, not the alert text — otherwise the model wastes its effort learning to echo input it’s always given. **Adapters on attention AND the feed-forward layers** — attention-only adapters learn the task but keep drifting on output format.

Evaluation — the part most projects skip

proving it works, honestly

What it is. Measuring the model against a held-out answer key rather than eyeballing a few outputs and calling it good.

The three levels — and the progression is the whole point:

- **Level 1, in-distribution.** Test on held-out examples with unseen service names. Result: 100% category accuracy.
- **Level 2, the suspicion.** 100% is a WARNING, not a trophy. That test reused the same 21 alert templates as training, so it can catch a model that memorised service names but NOT one that memorised the templates themselves.
- **Level 3, out-of-distribution.** A harder test: hand-written alerts sharing no template with training, plus adversarial cases where the obvious clue points at the WRONG answer.

The honest finding

On realistic unseen alerts the fine-tune jumped 40% → 60%. On the adversarial cases it moved **not at all** (50% → 50%). Conclusion: it learned better pattern-matching, not causal reasoning — exactly what template-generated training data should be expected to produce. Reporting this, instead of just the 100%, is the strongest thing in the project.

Prompt-injection defence

when the data tries to give orders

The threat, concretely. Anyone who can write a log line can write text the agent will later read. A customer types a weird username; it lands in a log; the investigator pulls that log into its context. If instruction-shaped text in a log can steer the agent, then anyone who can cause an error can steer production changes.

Analogy. A forged note slipped into the evidence pile saying “Detective: ignore everything else, arrest this man.” A good detective treats evidence as evidence, never as instructions.

Three layers. Neutralise instruction-like text before it reaches the prompt (replace it — never quote it back, or you’ve just repeated the attack), redact secrets and personal data, and keep the approval gate structural.

The insight that ties it together

Detection is best-effort and will eventually miss something. The gate **does not depend on detection working**. Even if an injection slips past every filter, it still cannot execute anything — a human is still in the way. Testing found the *polite* attack (“the on-call engineer already approved this”) slipped past first; the shouty “IGNORE ALL INSTRUCTIONS” was easy to catch.

11

Model routing by tier

the right model for each job

What it is. Nodes ask for a *tier* (“planner”, “worker”) — not a specific model name. A config file decides which real model each tier points to.

Analogy. A company org chart. You send a task to “the finance team,” not to a named person. Management can swap who fills the role without every task breaking.

Why it matters. It’s the seam that makes the fine-tuning result *measurable* — same test, same inputs, swap one config line to compare the old model against the new. It’s also cost engineering: a cheap local model classifies, an expensive one plans.

12

Observability

seeing inside a run

What it is. Recording a detailed trace of everything the agent did — every model call, every tool call, with prompts, timings and token counts — so you can inspect it afterwards (we use LangSmith).

Analogy. A flight recorder. When something goes wrong you don’t want a one-line summary; you want the full timeline of every input and decision.

Why traces beat logs. When the critic rejects a hypothesis three times and the run burns 130,000 tokens, the question is *which* node, on *which* loop, with *what* prompt. That’s a question about a tree, not a flat log line.

A real trap we hit

The settings loader reads the config file into an object, but the tracing library reads raw environment variables — two different places. Flipping the switch in the config file looked enabled, reported itself enabled, and traced nothing. The fix verifies the tracer actually started, instead of trusting the flag.

The Results — and the Honest Story

The fine-tuned model vs the baseline

Both models were given the exact same held-out alerts. “Baseline” is a general 3-billion-parameter model just prompted with instructions; “fine-tuned” is our 1.5-billion model trained for this one job — half the size, better numbers, half the latency.

Metric	Baseline	Fine-tuned
Valid JSON (parseable output)	often failed	100%
Severity accuracy	37.5%	97.5%
Category accuracy	25.0%	100%
Critical misses (P1 seen as minor)	10.0%	0%
Latency (p50)	2980 ms	1622 ms

Read the “valid JSON” row first

The baseline’s category accuracy was low largely because it couldn’t reliably produce parseable output at all — and a classifier whose answer won’t parse is a broken classifier, no matter what it ‘knew.’ Getting a small model to 100% clean structured output is the quiet, real win.

The test that told the truth

A 100% score should make you suspicious, not proud. That test reused the same 21 alert templates as the training data — so it can’t tell “learned the task” apart from “memorised the templates.” So we built a harder test: brand-new alerts sharing no template, plus adversarial ones where the obvious surface clue points at the wrong answer.

Generalization test	Baseline	Fine-tuned
Realistic unseen alerts	40%	60% (+20)
Adversarial (traps)	50%	50% (+0)

This is the headline finding — lead with it, don’t hide it

Big gains on realistic new alerts; **zero movement** on cases built to fool surface-level pattern matching. That means the model learned to match patterns better, but did *not* learn the underlying cause-and-effect reasoning — precisely what you’d expect from template-generated data. “I got 100%, found it suspicious, built a harder test, and proved the model had memorised patterns” shows scientific judgment. That sentence is worth more than the 100% ever was.

Why the adversarial score didn’t move — and how to fix it

The training data had no *causal* variety: every “resource-exhaustion” example literally contained “exit code 137,” so the model learned the *string*, not the *concept*. The fix is a data generator that varies the mechanism (same cause, different symptoms; same symptoms, different cause) rather than just the surface words. Naming this yourself, before anyone asks, is more impressive than the perfect score.

How to Explain It

Three résumé bullets

Bullet 1 — the system

Built **Sentinel**, an autonomous incident-response agent using LangGraph and LangChain — a 10-node state machine with a cyclic self-critique loop, Postgres checkpointing, and a human-in-the-loop interrupt gating all destructive actions. Hybrid RAG (BM25 + pgvector, fused via reciprocal rank fusion) grounds investigations in a runbook corpus, and a custom MCP server exposes the toolset to any protocol-compatible client.

Bullet 2 — the fine-tune

Fine-tuned **Qwen2.5-1.5B with QLoRA** for alert triage, lifting category accuracy from 25% → 100% and severity accuracy 37.5% → 97.5% while halving p50 latency (2980ms → 1622ms) on a model half the baseline's size; eliminated critical-severity underestimates entirely (10% → 0%).

Bullet 3 — the judgment (your strongest)

Designed a **three-tier evaluation harness that exposed the model's limits rather than hiding them** — an out-of-distribution suite with adversarial cases showed the fine-tune gained +20pp on unseen realistic alerts but 0 on cases engineered to defeat surface-feature matching, proving it learned pattern-matching, not causal reasoning. Also built a 31-check red-team suite for prompt injection, PII redaction, and privilege escalation.

The tech stack, grouped

Agent & orchestration	LangGraph · LangChain · MCP (FastMCP) · Pydantic
Models & training	Qwen2.5-1.5B (QLoRA) · Ollama · OpenRouter · PEFT · TRL · bitsandbytes · llama.cpp/GGUF
Data & retrieval	PostgreSQL + pgvector · BM25 (rank-bm25) · Reciprocal Rank Fusion
Backend	Python 3.11 · FastAPI · SSE streaming · asyncio · httpx · psycpg3
Frontend	Streamlit (5-page console)
Observability & eval	LangSmith · structlog · custom eval harness (macro-F1, confusion matrices)
Infra & tooling	Docker (multi-stage, non-root) · Compose · GitHub Actions CI · uv · pytest (83 tests) · ruff

Interview Q&A — the questions you WILL get

Q “Walk me through what happens when an alert fires.”

A Triage classifies it; retrieve pulls relevant runbooks; an investigator with read-only tools gathers logs, metrics and deploys; synthesize forms a hypothesis; a critic attacks it and can loop back for more evidence; once solid, plan writes remediation steps; the graph PAUSES for human approval; on approval it executes, verifies recovery, and writes a post-mortem to memory.

Q “Isn’t the environment fake? Does it count?”

A Fake on purpose. A simulated world with a known ground truth is the only way to actually SCORE whether the agent found the right cause. Real infrastructure has no answer key. The tool interfaces match real ones (Prometheus, logs, k8s), so swapping in real backends is plumbing, not redesign.

Q “How do you stop the agent doing something destructive?”

A Three structural walls, not a polite request: the graph physically halts before execution; the approval flag is set outside the model’s reach; and the investigator is handed a read-only tool list so it has no destructive tool to call. Confidence is never authorization.

Q “Your model hits 100% — isn’t that overfitting?”

A Yes, and I caught it. The held-out test reused training templates, so I built an out-of-distribution suite with adversarial cases. The fine-tune gained on realistic new alerts but zero on the traps — it learned patterns, not causal reasoning. The fix is training data with real causal variety.

Q “Why fine-tune instead of just prompting a bigger model?”

A Triage is narrow, high-volume, and fixed-output — the textbook case for fine-tuning. A 1.5B specialised model beat a 3B general one on accuracy at half the latency, and runs locally for free. You wouldn’t fine-tune for the open-ended planning step — that stays on a bigger model.

Q “What was the hardest bug?”

A A dtype mismatch during fine-tuning: the model loaded in bf16 while training ran fp16, because a library renamed a parameter and silently ignored the old one. It only surfaced minutes into a training run. The fix picks precision from the actual hardware and ASSERTS the dtype applied instead of trusting it.

Glossary — Every Term in One Place

Agent	An AI system that doesn't just talk but takes actions — calling tools, deciding its own next step, looping until done.
Agentic loop	The core cycle: model requests a tool → you run it → feed the result back → repeat until the model stops asking.
Tool calling	The mechanism by which a model asks your code to run a named function with arguments.
LangChain	A library of building blocks for LLM apps — models, tools, prompts, retrievers — with one common interface.
LangGraph	A library for building agents as graphs (nodes + edges + shared state), supporting branches, loops and pauses.
Node / Edge	A node is one step (a box); an edge is a transition (an arrow) between steps.
State	The shared data object that flows through the graph and accumulates results as it goes.
Reducer	A merge rule on a state field. "Add" means new values append; without one, they overwrite.
Checkpointing	Saving the graph's state to a database at each step, so a run can pause and resume — even after a restart.
Interrupt	A configured point where the graph halts and waits (here: for human approval) before continuing.
Human-in-the-loop (HITL)	A design where a person must approve certain steps before the system proceeds.
Structured output	Forcing model output into a strict schema (typed fields) rather than free-form prose.
Schema / Pydantic	A definition of the exact shape data must take. Pydantic is the Python library that enforces it.
RAG	Retrieval-Augmented Generation — fetch relevant documents and add them to the prompt before the model answers.
Embedding / Vector	A list of numbers representing a text's meaning, so similar meanings sit close together.
Vector search	Finding documents by meaning-similarity rather than exact words.
BM25	A classic keyword-ranking algorithm — great at exact rare tokens that embeddings blur.
Hybrid search / RRF	Combining keyword and vector search. Reciprocal Rank Fusion merges their rankings without rescaling.
pgvector	A PostgreSQL extension that stores embeddings and does vector search inside the database.
Chunking	Splitting documents into pieces for retrieval. We split on headings, keeping each piece coherent.
Critic / reflection	A second agent that attacks the first's output; a loop back for more work if it's weak.

Multi-agent	Multiple specialised agents (investigator, critic, planner) each with their own prompt and tools.
MCP	Model Context Protocol — an open standard for exposing tools/data to any AI system. “USB-C for AI tools.”
MCP server / client	A server publishes tools; a client consumes tools published by others.
Fine-tuning	Specialising a model for one task by training it on task-specific examples.
LoRA	Low-Rank Adaptation — freeze the base model, train tiny adapter matrices (~1% of parameters).
QLoRA	LoRA with the base model quantised to 4-bit, so a bigger model fits on a small GPU.
Quantisation	Storing model weights in lower precision (e.g. 4-bit) to save memory.
Adapter	The small set of trained weights LoRA adds on top of the frozen base model.
GGUF / Ollama	GGUF is a portable model file format; Ollama is a tool that runs such models locally.
Baseline	The comparison point — here, a general model just prompted, before fine-tuning.
Held-out / OOD	Held-out data isn't used in training. OOD (out-of-distribution) is deliberately unlike training data.
Macro-F1	An accuracy metric that treats every class equally, so rare classes still count.
Confusion matrix	A grid showing what got predicted vs what was true — the diagonal is correct.
Prompt injection	An attack where malicious instructions hidden in data try to hijack the model.
Blast radius	How much damage an action can do — used to decide what needs approval.
Model routing / tier	Sending each task to a role (“planner”), with config deciding the actual model.
Observability / tracing	Recording a full trace of a run (every call, prompt, token) for later inspection.
Token	The unit models read and write text in — roughly a word-piece; usage is measured in tokens.
Latency (p50/p95)	Response time. p50 = median; p95 = the slow tail (95% are faster than this).
SSE	Server-Sent Events — a way to stream updates to a browser as they happen.

One last thing to internalise

The strongest signal in this whole project is not any single technology — it's that you measured your own work honestly, found where it fell short, and could explain exactly why. Anyone can wire up a demo. Knowing what your demo does NOT prove is the senior skill.